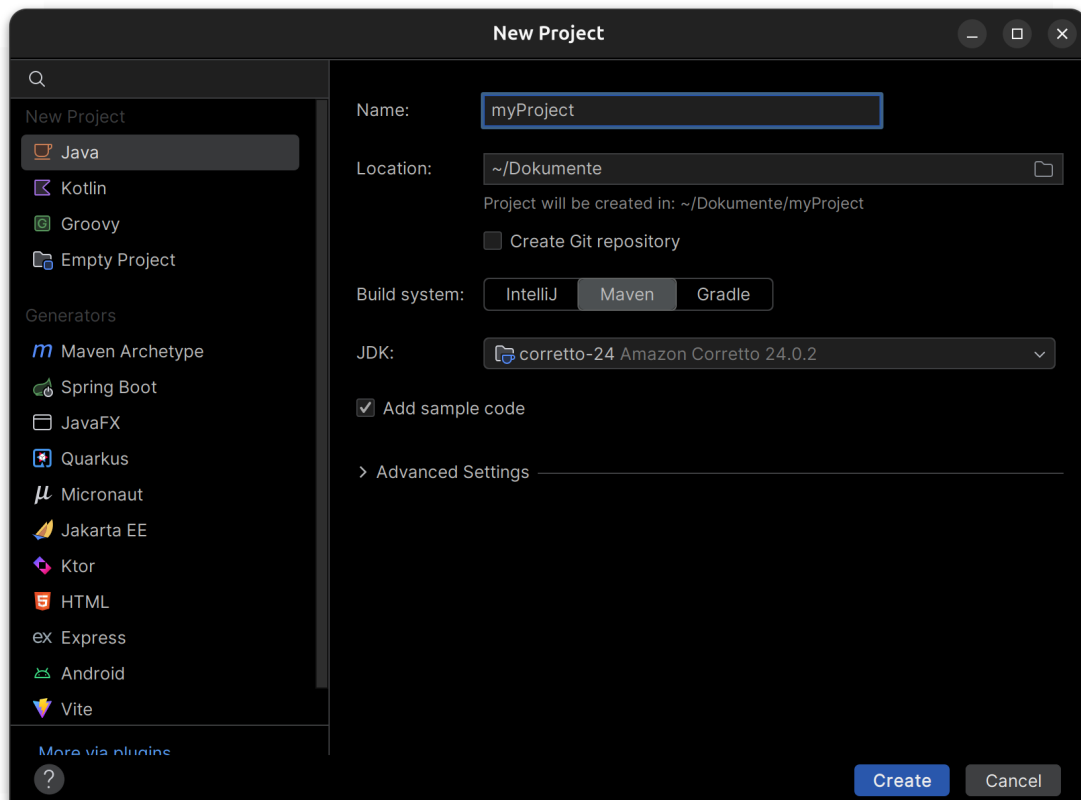


Java (für SuS mit Kotlin-Kenntnissen)

Um alle Features, die hier erklärt werden, nutzen zu können musst du die JDK 24 und Maven nutzen.



Außerdem musst du die Preview-Features aktivieren. Dies geht am einfachsten, wenn du ein Preview-Feature verwendest.

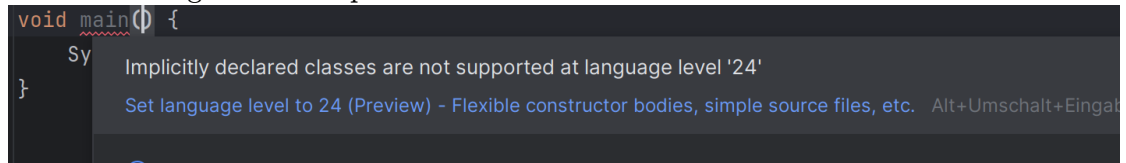
Vor Version 24 musste man in Java eine Main-Klasse mit main-Methode schreiben. Z.B.

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Ab Java 24 reicht eine einfache main-Funktion aus.

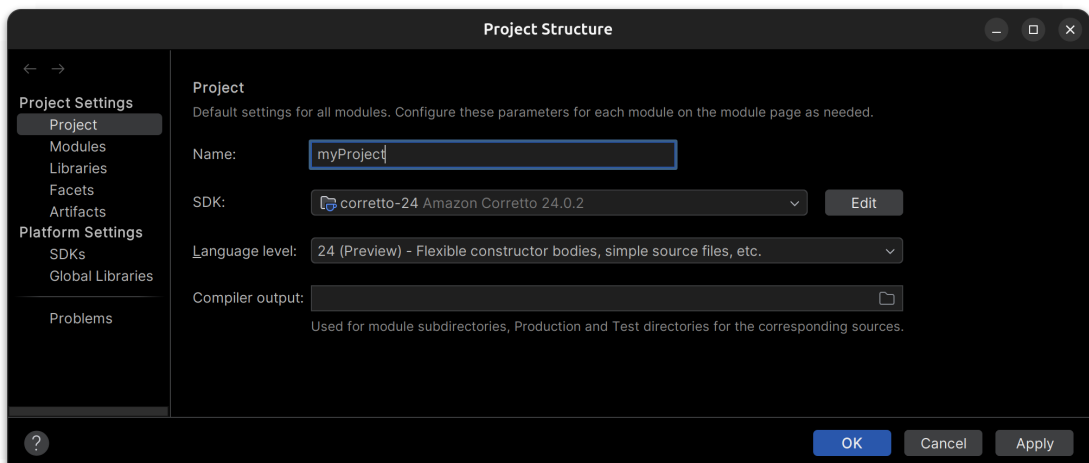
```
void main() {  
    System.out.println("Hello, World!");  
}
```

Beim hovern über diese Funktion wird dir vorgeschlagen, die Preview-Features zu aktivieren. Dies geht einfach per Klick auf den blauen Text.



1 Projekteinstellungen ändern

Wenn die Projekteinstellungen nicht mehr wie oben gesetzt sind, kannst du auf File und Project Structure klicken und die Einstellungen wieder korrigieren.



2 Variablen

2.1 Mit Typangabe

Beim Initialisieren einer Variable steht der Typ vor der Variablen.

```
1 int a = 2;  
2 a = a + 1;  
3 a;
```

3

2.2 Ohne Typangabe

Auf die Typangabe kann auch verzichtet werden, wenn stattdessen das Schlüsselwort `var` genutzt wird.

```
1 var b = true;  
2 b && false;
```

false

3 Primitive und nicht primitive Typen

In Java gibt es neben den primitiven Typen wie `int` und `boolean` entsprechende Klassen wie `Integer` und `Boolean`.

```
1 Integer c = 5;
```

primitiven Typen	Klasse
int	Integer
boolean	Boolean
char	Character
float	Float
double	Double
	String

Im Gegensatz zu den primitiven Datentypen können wir auf Objekten dieser Klassen Methoden nutzen.

```
1 String d = c.toString();
```

Der Vergleich mit `==` prüft bei primitiven Typen, ob zwei Werte inhaltlich gleich sind.

```
1 int e = 3;  
2 int f = 3;  
3 e == f;
```

true

Bei Objekten wird mit `==` geprüft, ob es sich um dasselbe Objekt handelt.

```
1 Integer g = 150;  
2 Integer h = 150;  
3 g == h;
```

false

Objekte können mit der equals Methode auf inhaltliche Gleichheit geprüft werden.

```
1 g.equals(h);
```

true

Primitive Datentypen und die entsprechenden Klassen werden automatisch konvertiert.

```
1 Integer i = 5;  
2 int j = i;  
3 j;
```

5

Im Gegensatz zu primitiven Typen können Werte von Klassen immer null sein.

```
1 Integer noInt = null;
```

4 Ausgabe an der Konsole

Die Ausgabe an der Konsole erfolgt mit System.out.println.

```
1 System.out.println("hello");
```

hello

5 Funktionen

Bei der Definition von Funktionen beginnt man mit dem Rückgabetyp. Ein Schlüsselwort für die Definition von Funktionen gibt es nicht. Die Typen der Parameter stehen vor dem Parameternamen.

```
1 int square(int x){  
2     return x * x;  
3 }
```

```
1 square(3);
```

9

Wenn nichts zurückgegeben wird, schreibt man void statt eines Rückgabetyps.

```

1 void printSquare(int x){
2     System.out.println(x * x);
3 }
4
5 printSquare(5);

```

25

6 Fallunterscheidungen

6.1 Zweiseitig

if - else funktioniert wie in Kotlin, kann aber nicht als Ausdruck genutzt werden.

6.2 Mehrseitig

Um einen Wert mit mehreren Möglichkeiten zu vergleichen verwendet man in Java das Schlüsselwort switch.

Es gibt switch-Ausdrücke

```

1 var k = 2;
2
3 var l = switch(k) {
4     case 1 -> "One";
5     case 2 -> "Two";
6     default -> "Big Number";
7 };
8
9 l;

```

Two

Der Wert des gesamten Ausdrucks ist der Wert des Ausdrucks, der rechts von dem Wert steht, mit dem der Ausdruck in Klammern übereinstimmt.

Es gibt auch switch-Statements.

```

1 var m = 3;
2 var result = "";
3
4 switch(m) {
5     case 1 -> result = "One";
6     case 2 -> result = "Two";
7     default -> result = "Big Number";

```

```
8   };  
9  
10  result;
```

Big Number

Hier wird der Code ausgeführt, der rechts von dem Wert steht, mit dem der Ausdruck in Klammern übereinstimmt.

7 While-Schleifen

Bei einer while-Schleife muss man in der Regel

- eine Variable initialisieren
- prüfen, ob der Schleifenkörper nochmal ausgeführt werden soll
- den Wert der Variablen ändern

```
1  var n = 1;  
2  while (n < 10){  
3      System.out.println(n);  
4      n = n * 2;  
5  }
```

```
1  
2  
4  
8
```

8 For-Schleifen

Bei einer for-Schleife benötigen wir in Java dieselben Bestandteile. Hier steht aber alles im Schleifenkopf.

```
1  for (var o = 1; o < 10; o = o * 2){  
2      System.out.println(o);  
3  }
```

```
1  
2  
4  
8
```

for- Schleifen werden oft genutzt, um in Einer-Schritten hochzuzählen.

```

1 for (var p = 1; p < 4; p++){
2     System.out.println(p);
3 }

```

```

1
2
3

```

9 Unveränderliche Objekte

9.1 Definition der Klasse und Erzeugung von Objekten

Mit dem Schlüsselwort `record` können relativ einfach Klassen mit unveränderlichen Eigenschaften erzeugt werden.

```

1 record Student(String name, int age){};

```

Vor jedem Aufruf des Konstruktors muss `new` stehen.

```

1 var alex = new Student("Alex", 18);

```

9.2 Zugriff auf Eigenschaften

Auf die Eigenschaften von Records kann nicht direkt von außen zugegriffen werden. Um den Zugriff zu ermöglichen, werden aber automatisch Methoden definiert, die genau wie die Eigenschaften heißen.

```

1 alex.age();

```

```

18

```

9.3 Methoden

Methoden werden zwischen geschweiften Klammern definiert.

```

1 record SpeakingStudent(String name, int age){
2
3     String generateGreeting(){
4         return "I am " + name + " and I am " + age + " years old!";
5     }
6
7 };

```

```

1 var speakingStudent = new SpeakingStudent("Alex", 18);
2 speakingStudent.generateGreeting()

```

I am Alex and I am 18 years old!

9.4 Sekundäre Konstruktoren

Im Klassenkörper können auch sekundäre Konstruktoren definiert werden.

Die Definition beginnt mit dem Namen der Klasse. Anschließend folgen in runden Klammern die Parameter des Konstruktors. In den geschweiften Klammern wird mit `this` der primäre Konstruktor aufgerufen.

```

1 record Student(String name, int age){
2
3     Student(String name){
4         this(name, 14);
5     }
6
7 };

```

```

1 var nino = new Student("Nino");
2 nino.age();

```

14

10 Listen

10.1 Unveränderliche Listen

Listen müssen zunächst importiert werden.

```

1 import java.util.List;

```

Mit `List.of` kann ein Element vom Typ `List` erzeugt werden.

```

1 var xs = List.of(1, 3, 5);

```

Mit einer `for`-Schleife können die Elemente in einer Liste durchlaufen werden.

```

1 for (var x : xs){
2     System.out.println(x);
3 }

```



```
1  
3  
5
```

Mit der Methode `get` kann auf Elemente in einer Liste zugegriffen werden.

```
1 xs.get(1);
```

```
3
```

Mit der Methode `size` kann die Anzahl der Elemente in einer Liste bestimmt werden.

```
1 xs.size();
```

```
3
```

10.2 Veränderliche Listen

Objekte der Klasse `ArrayList` sind veränderliche Listen.

```
1 import java.util.ArrayList;
```

Wenn dem Konstruktor nichts übergeben wird, wird eine leere Liste erzeugt. In diesem Fall muss der Typ in eckigen Klammern angegeben werden. Für den Typ der Elemente einer Liste können wir keinen primitiven Typ wählen.

```
1 var ys = new ArrayList<Integer>();  
2 ys;
```

```
[]
```

Es kann aber auch eine Liste übergeben werden.

```
1 var zs = new ArrayList<>(List.of(1, 3, 5));  
2 zs;
```

```
[1, 3, 5]
```

Auf einer `ArrayList` kann man die Methode `add` aufrufen, die ein Element anhängt.

```
1 zs.add(7);  
2 zs;
```

```
[1, 3, 5, 7]
```

Mit der Methode `set` kann man das Element an einer gewünschten Stelle ersetzen.

```
1 zs.set(2, 10);
2 zs;
```

[1, 3, 10, 7]

10.3 Array-Listen als Listen verwenden

Immer wenn der Compiler einen Wert vom Typ List erwartet, können wir auch eine ArrayList übergeben.

```
1 List<Integer> firstTwo(List<Integer> xs){
2     var ys = new ArrayList<Integer>();
3     ys.add(xs.get(0));
4     ys.add(xs.get(1));
5     return ys;
6 }
7
8 var emptyArrayList = new ArrayList<>(List.of(1, 3, 5));
9 firstTwo(emptyArrayList);
```

[1, 3]

11 Typparameter

Typparameter werden vor dem Rückgabetyt definiert.

```
1 <T> T firstElem(List<T> xs){
2     return xs.get(0);
3 }
```